Your pipeline ran.
Your tests passed.
Your numbers were 10× wrong.

Here is how that happens, and how you catch it.

Retail Lakehouse Pipeline · Mohamed Taha Abo Heiba

PROJECT SCOPE

From fragmented retail data to trusted decisions

1

Sources

- **PostgreSQL**: 6 OLTP tables
- **AWS S3**: monthly supplier CSVs

2

Pipeline

Databricks lakehouse · dbt · Airflow · GitHub Actions CI

3

Consumption

- Power BI 3-page report
- Databricks Genie Agent

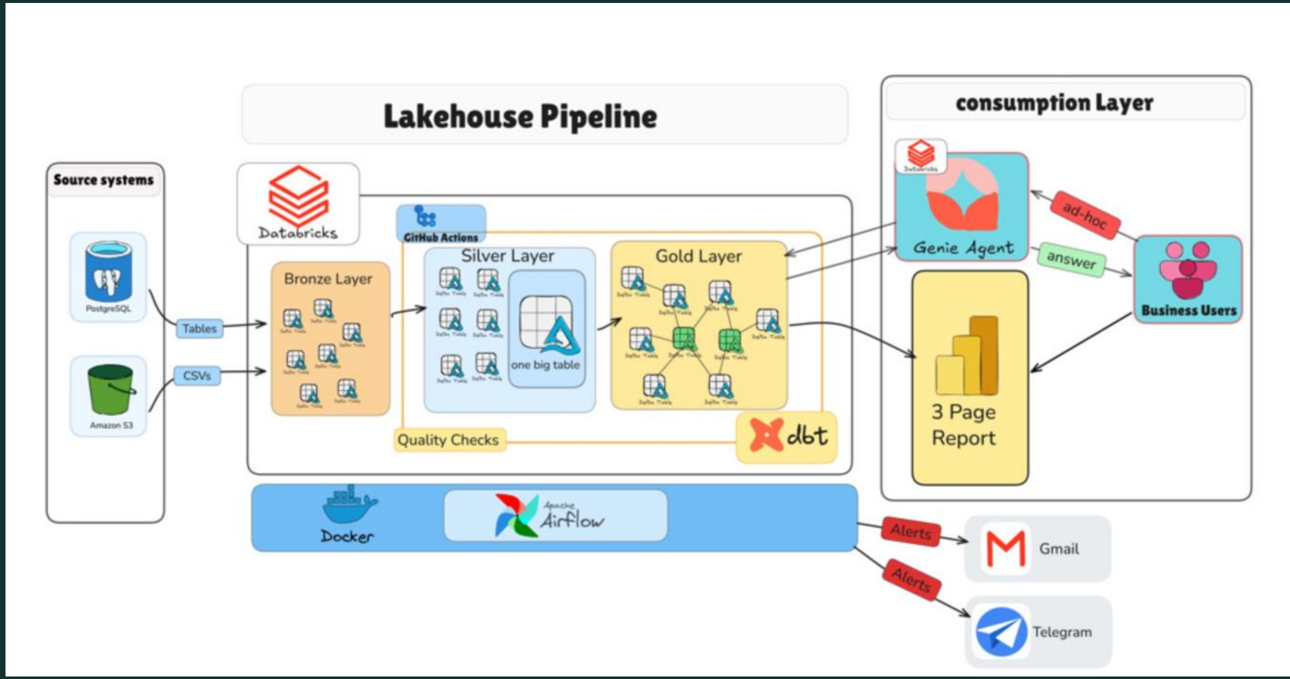
At a glance

- **123/123** dbt tests passing
- **Galaxy schema** gold layer
- **Built for** sales, Supplies, and workforce analysis
- **Dual-surface** consumption layer

Goal: Turn fragmented retail operational data into a **trusted analytical layer** for sales, procurement, and workforce decisions.

ARCHITECTURE

Two sources. Three layers. Two consumption surfaces.



Full-width architecture diagram — insert your own screenshot here

PostgreSQL and S3 enter through independent bronze paths, dbt transforms them, and become a trusted gold layer consumed by Power BI and Genie Agent.

Click here for Full walkthrough

INGESTION

Two ingestion paths. One bronze layer.

Primary Path — PostgreSQL

Lakeflow Connect · cursor column + primary key upsert per table

CDC unavailable in this setup — downstream SCD2 design accounts for the limitation.

Click here for PostgreSQL details

Secondary Path — AWS S3

Supplier CSVs · managed Lakeflow pipeline (no custom code)

Airflow verifies terminal state before downstream runs

Click here for S3 details

TRANSFORMATION

The pipeline was green.
The data was wrong.

7 incremental silver models

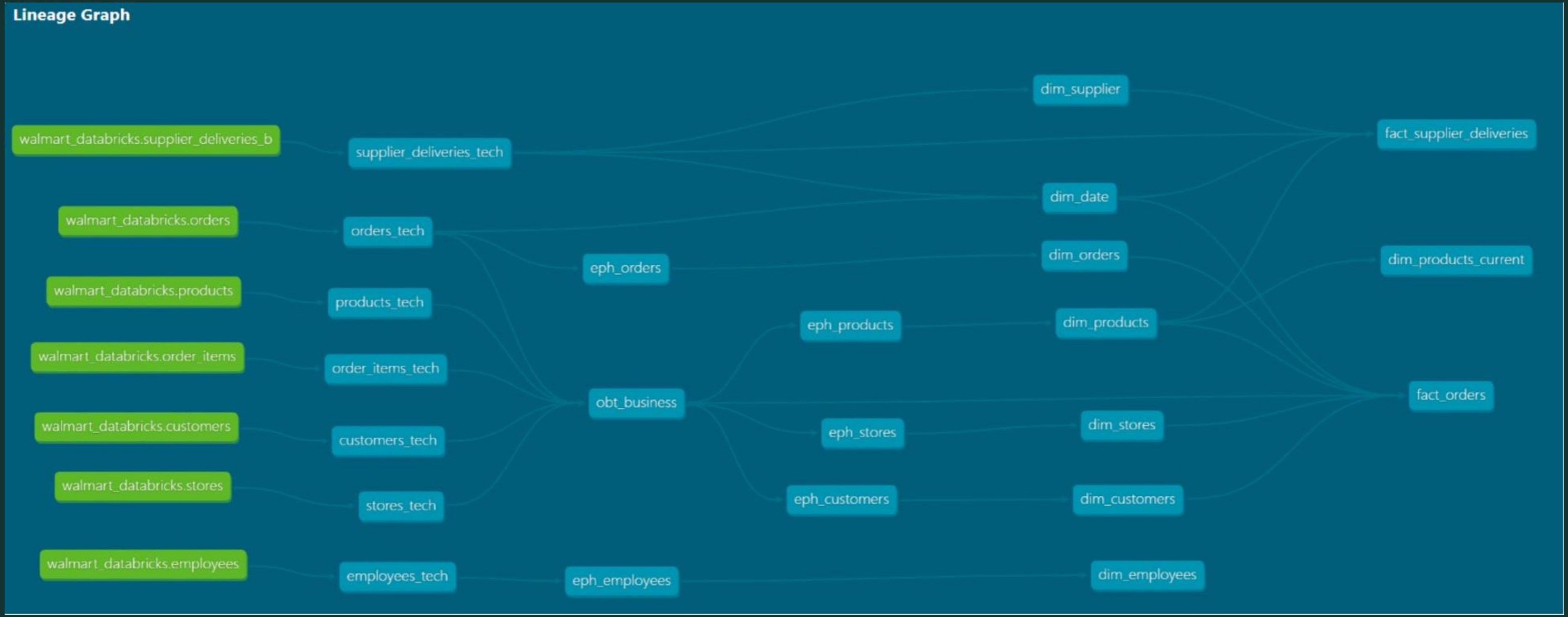
SCD2 snapshots across 4 dimensions

Metadata-driven wide table
One config entry per source — no hand-written SQL per join

obt_business returned **300,513 rows.**
Expected: **30,021.**
Every dbt test passed clean.

Root cause: employees were joined through `store_id`, creating a one-to-many fan-out.

Fix: removed the invalid relationship and added explicit grain validation.



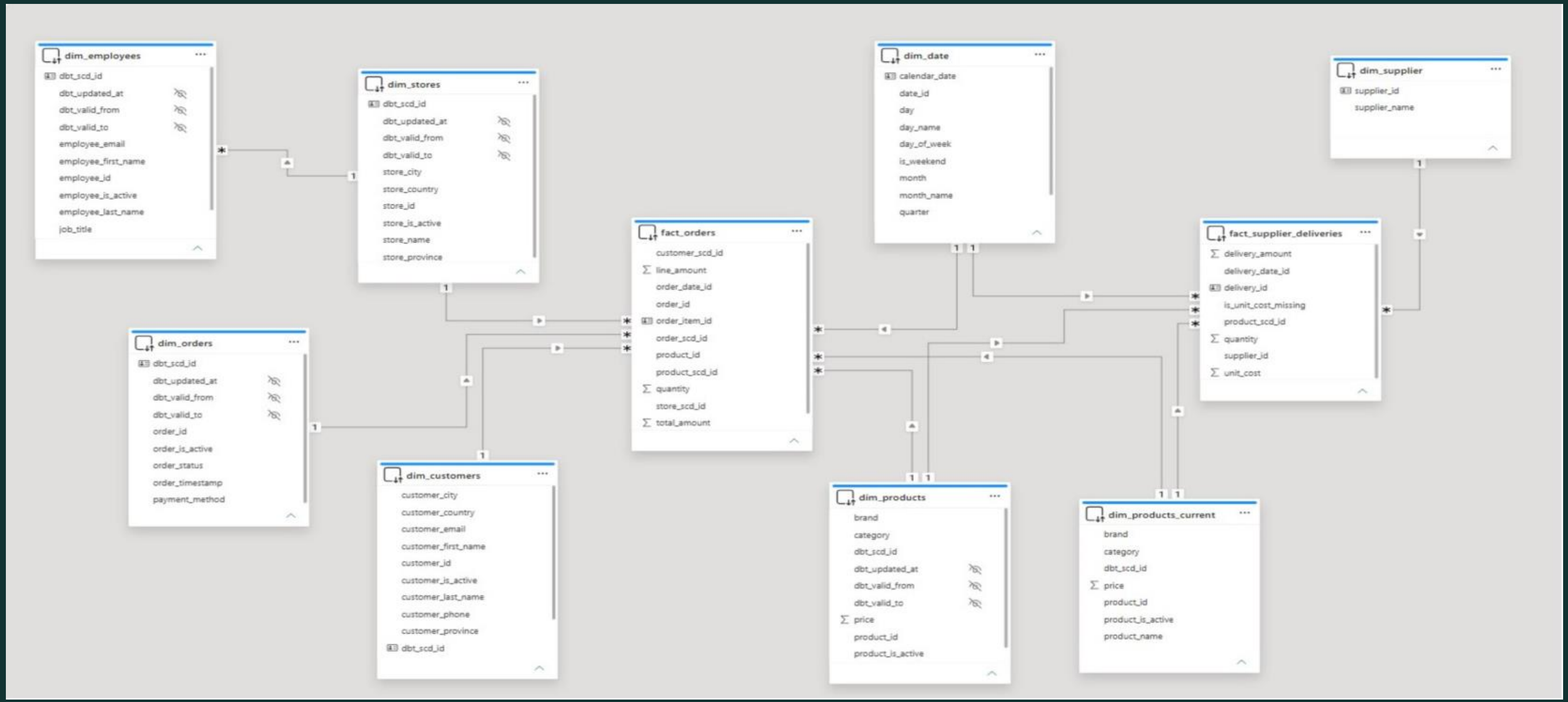
dbt lineage diagram

Click here for dbt details

Made with GAMMA

GOLD LAYER

Galaxy schema. Two business processes.Shared dimensions only where the grain supports them.



Galaxy schema ERD

fact_orders
Sales · order line grain

fact_supplier_deliveries
Procurement · delivery grain

Shared Dimensions
`dim_date` and `dim_product` shared across both facts

Click here for Gold Layer details

ORCHESTRATION

Sequential vs Parallel branches

Measured improvement. Two alert channels.

9:03

Sequential avg

-16%

Runtime reduction
across 3 measured runs

7:35

Parallel avg

Alerting

Task failure → **Gmail + Telegram**

Click here for Orchestration details

TESTING

Green tests **do not** guarantee correct data.

Referential integrity checks

across dimension relationships

Singular Grain Checks

for every fact table — catching the 10× fan-out class of bug

CI Quality Gate

Every push · blocks merge on failure

```
Terminal                                                              powershell
(base) (taha) PS C:\Users\ORIGINAL\Desktop\dbt> dbt test
21:42:01  112 of 123 PASS unique_dim_products_dbt_scd_id ................................ [PASS in 9.74s]
21:42:01  122 of 123 START test unique_stores_tech_store_id .............................. [RUN]
21:42:03  113 of 123 PASS unique_dim_stores_dbt_scd_id .................................. [PASS in 12.30s]
21:42:03  114 of 123 PASS unique_dim_supplier_supplier_id ............................... [PASS in 11.83s]
21:42:03  115 of 123 PASS unique_employees_tech_employee_id ............................. [PASS in 11.83s]
21:42:03  123 of 123 START test unique_supplier_deliveries_tech_delivery_id .............. [RUN]
21:42:07  117 of 123 PASS unique_fact_supplier_deliveries_delivery_id ................... [PASS in 9.56s]
21:42:08  116 of 123 PASS unique_fact_orders_order_item_id .............................. [PASS in 10.79s]
21:42:08  119 of 123 PASS unique_order_items_tech_order_item_id ......................... [PASS in 10.78s]
21:42:08  118 of 123 PASS unique_obt_business_order_item_id ............................. [PASS in 10.79s]
21:42:12  120 of 123 PASS unique_orders_tech_order_id ................................... [PASS in 12.20s]
21:42:13  122 of 123 PASS unique_stores_tech_store_id ................................... [PASS in 11.89s]
21:42:13  121 of 123 PASS unique_products_tech_product_id ............................... [PASS in 12.56s]
21:42:13  123 of 123 PASS unique_supplier_deliveries_tech_delivery_id ................... [PASS in 9.47s]
21:42:13
21:42:13  Finished running 123 data tests in 0 hours 3 minutes and 45.56 seconds (225.56s).
21:42:14
21:42:14  Completed successfully
21:42:14
21:42:14  Done. PASS=123 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=123
```

A green dbt run confirms execution. It does not confirm correctness.

Click here for testing details

CONSUMPTION

Fixed report for the known questions. Agent for everything else.

Power BI

3 pages: **Sales** · **Supplies** · **Workforce**

Designed for recurring operational questions

Import mode via Databricks Partner Connect

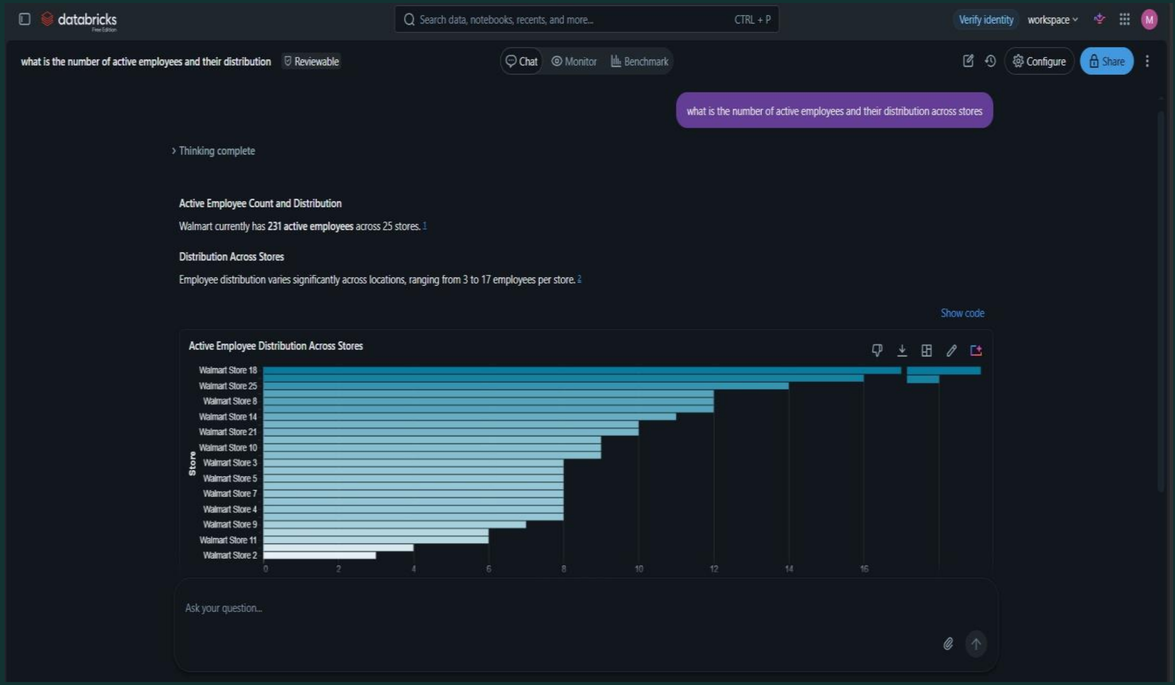
Genie Agent

Natural-language exploration across the gold layer

Designed for questions beyond the predefined report

Table and column descriptions pre-annotated in Unity Catalog

Power BI Workforce page



Genie Agent

Not redundant. Power BI answers recurring questions. Genie explores the questions nobody wrote a query for yet.

Click here for Business layer details

Live report

CLOSING

This project is Built to be trusted, explained, and challenged, not just demonstrated.

Every decision has a reason

- Galaxy vs star schema
- Import vs DirectQuery
- Two alert channels vs one

Every claim has a number

- 16% DAG improvement
- 123/123 tests
- 10× row-count fan-out
- 3-run benchmark

Every limitation is documented

- No CDC, by design
- No employee-to-order link in source
- Report freshness bounded by refresh schedule

LET'S TALK

Thanks for making it this far.

If you want to talk data engineering, architecture, or building pipelines that can be trusted, I'd like to hear from you.

Beyond Building the Pipeline

This project taught me that building a pipeline is **only half the job**. The harder part is proving that the data is correct, understanding the **trade-offs** behind every design decision, and being honest about what the system cannot guarantee.

I built this project to demonstrate how I think as a Data Engineer: design for scale, test for correctness, measure what matters, and document the decisions

---

REACH OUT

Portfolio GitHub LinkedIn

Have a problem worth solving? Let's talk.

Mohamed Taha Abo Heiba · Data / AI Engineer